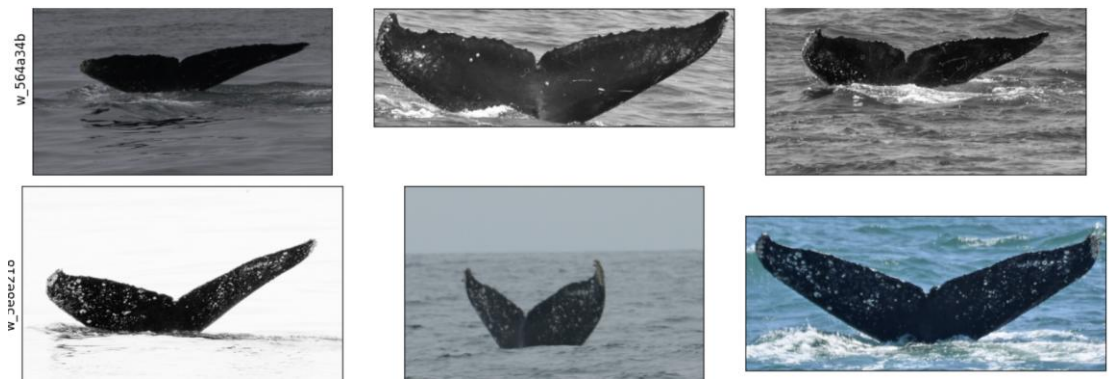


Assignment 1 – Practical Deep Learning Workshop

Part 1: exploratory data analysis

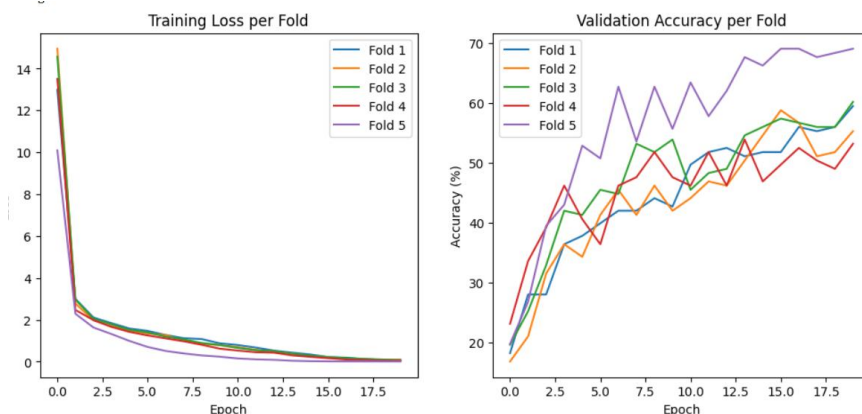
- Total number of training images: 25361
Total unique whale classes (IDs): 5005
- Most common whale: new_whale with 9664 images
Least common whale: w_6cb0480 with 1 images
Each sample contains an image and a label. The images have different sizes so we should resize them to be in the same size. The images were taken from different angles and in different lighting so we can augment the data, the model should work well on the augmented data.
- As we can see above, the dataset is extremely imbalanced, as there's almost 10000 whales as "new whale" which means they are unidentified, while specific whales have only a couple of images because of that we will use the 15 classes with the most samples excluding new whale.
- In the Kaggle leaderboard there are contestants that have achieved a score of about 97% accuracy they used transfer learning with the senet154 model and they did 4 fold cross validation.
-



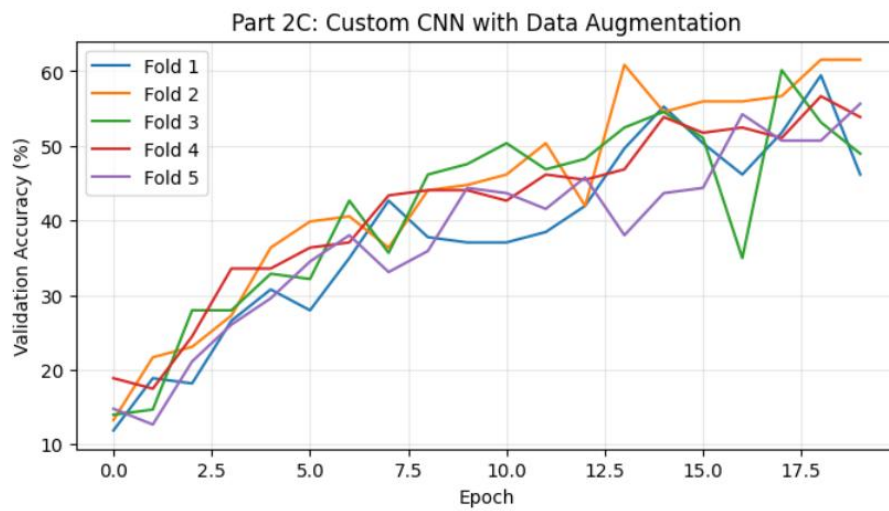
Part 2: cnn model

- The model takes RGB images resized to 128 x 128 pixels as input and consists of 3 Convolutional Blocks followed by 2 Fully Connected (Dense) Layers. We did 5fold validation and used 20 epochs

Results:

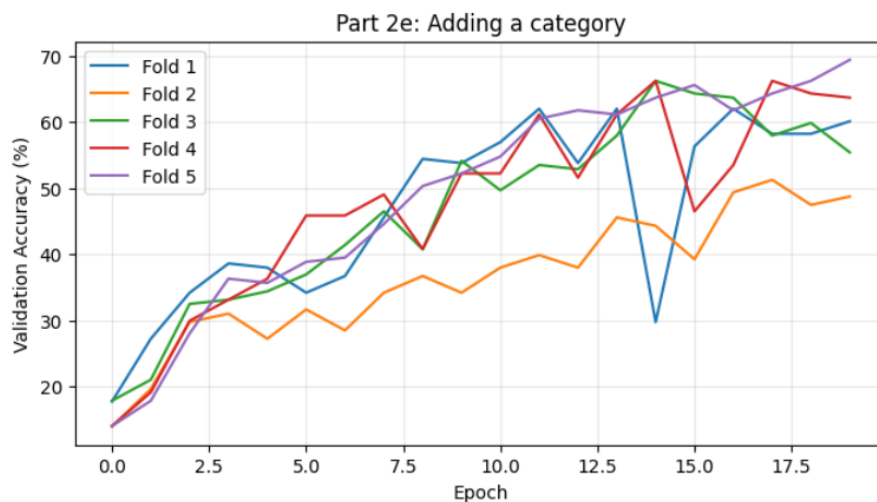


- b. There are several reasons why the model misclassifies images
 1. In some images the lighting is less good
 2. In some images the tail is very small compared to the ocean
 3. Some of the different tail types are very similar
 4. The tails of the whale are sometime shown from different angles
 - c. 3 ways to improve our model (Prioritized)
 1. We can add RandomRotation library to help with tilted tails
 2. We can use ColorJitter library to help with the different lighting of the images
 3. We can crop the images if the tail is only in a small part of the image
- Results of using random rotation and color jitters libraries:



Using the augmentation libraries did not improve our accuracy and we believe that it is due to the network being too small with not enough epochs

- d. INFERENCE TIME AUGMENTATION - For each test image, we generated 5 augmented versions (random crops/rotations), predicted the class for each, and averaged the probabilities that did not improve our model
 - e. Adding category w_23a388d to the data
- Results:



Part 3:

- We compared four major architectures: ResNet18, DenseNet121, MobileNetV2, and VGG16. We used 2-Phase Training (Phase 1: Train only the head for 5 epochs; Phase 2: Fine-tune all layers for 5 epochs).
- We augmented the data, we resized and normalized the images
-

FINAL MODEL RESULTS:

	model	num_parameters	val_loss	val_accuracy	test_loss	test_accuracy	unique_correct	unique_errors
0	resnet18	11184207	0.246215	95.035461	0.314990	91.139241	72	7
1	densenet121	6969231	0.189568	97.183099	0.282205	91.139241	72	7
2	mobilenet_v2	2243087	0.394751	92.957746	0.522938	83.544304	66	13
3	vgg16	134321999	0.376642	94.366197	0.409124	88.607595	70	9

- We used pre-trained DenseNet121 to extract the features we reduced the dimensionality to 50 using pca and we used the random forest classifier. The mean accuracy of the result was 71.28%
-

	Experiment #	val-Loss	val-Accuracy	Optimizer	augmentation
0	resnet18	0.246215	95.035461%	Adam	Resize to 224, 224 and normalize
1	densenet121	0.189568	97.183099%	Adam	Resize to 224, 224 and normalize
2	mobilenet_v2	0.394751	92.957746%	Adam	Resize to 224, 224 and normalize
3	vgg16	0.376642	94.366197%	Adam	Resize to 224, 224 and normalize
4	random forest with feature extraction	random forest doesn't use a loss function	71.28%	random forest doesn't use an optimizer	Resize to 224, 224 and normalize

Part 4: summary

At the beginning we made a simple CNN architecture with 5fold validation and 20 epochs this led to around 60% accuracy. We later tried to augment some of the samples in the train set using RandomRotation and ColorJitter libraries and that did not improve our model. We later implemented inference-time-augmentation which improved our model by approximately 2%, which was not a big improvement. We assume that the augmentation did not improve our model because it is too small. We then used transfer learning with 4 models resnet18, densenet121, mobilenet_v2, vgg16 this improved our model by over 30%, the accuracies were between 90% and 97% this led to the conclusion that these models that were trained with many images and with a lot of parameters can extract certain features of an image so even if we only change the last layer. The machine learning model that we did with the features extracted with densenet121 had an accuracy of 73.69% which is still better than the CNN model we did from scratch.